

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Dipartimento di Informatica – Scienza e Ingegneria

Corso di Laurea Magistrale in
Informatica

# PoS tagging italiano con BERT

Progetto di Natural Language Processing

**Studente:** Pierre Errico

**Matricola:** 0001235580

---

Anno Accademico 2026/2027

# Abstract

Il Part-of-Speech (PoS) tagging è un compito di etichettatura di sequenze nel quale a ogni token viene assegnata una categoria grammaticale. Il presente lavoro si propone di affrontare il PoS tagging per la lingua italiana utilizzando un modello BERT preaddestrato, e confronta tre configurazioni: encoder congelato con classificatore lineare, fine-tuning completo con classificatore lineare e fine-tuning completo con l'aggiunta di un Conditional Random Field (CRF).

L'obiettivo è valutare i risultati prodotti da ciascuna configurazione e confrontarli con quelli del sistema di riferimento UDPipe 2, utilizzando come benchmark UD Italian-ISDT 2.15, un treebank annotato secondo le linee guida Universal Dependencies.

# Indice

**1 Introduzione** **1**
1.1 Il compito di PoS tagging . . . . . 1
1.2 Le principali sfide del PoS tagging . . . . . 2
1.3 Modelli neurali per l'elaborazione del linguaggio . . . . . 2
1.4 Architettura BERT . . . . . 3
1.4.1 Tokenizzazione . . . . . 3
1.4.2 Struttura dell'encoder . . . . . 3
1.4.3 Pre-addestramento . . . . . 4
1.5 Obiettivi del progetto . . . . . 5

**2 Metodologia** **6**
2.1 Disegno sperimentale . . . . . 6
2.2 Dati e annotazioni . . . . . 7
2.2.1 Corpus e suddivisione . . . . . 7
2.2.2 Rappresentazione di UPOS e FEATS . . . . . 7
2.3 Preprocessing e allineamento . . . . . 8
2.4 Modelli e funzione obiettivo . . . . . 8
2.5 Procedura di addestramento . . . . . 9
2.6 Selezione del checkpoint . . . . . 9
2.7 Valutazione . . . . . 10
2.7.1 Metriche UPOS . . . . . 10
2.7.2 Metriche FEATS . . . . . 10
2.7.3 Aggregazione e analisi degli errori . . . . . 11
2.8 Confronto con UDPipe 2 . . . . . 11
2.9 Riproducibilità e integrità delle esecuzioni . . . . . 12

**Bibliografia** **13**

# Capitolo 1

# Introduzione

## 1.1 Il compito di PoS tagging

Il PoS tagging è un problema di etichettatura di sequenze di token, nel quale a ogni token viene assegnata un'etichetta che ne identifica la categoria grammaticale nel contesto frasale. Un token corrisponde generalmente a una parola; può tuttavia indicare anche altri elementi della frase, come simboli e punteggiatura. Il PoS tagging è un compito fondamentale per l'analisi linguistica e costituisce una fase preliminare per numerose applicazioni di Natural Language Processing (NLP), tra cui la lemmatizzazione, l'analisi sintattica e la traduzione automatica [1, 2].

Nel sistema Universal Dependencies (UD), l'inventario UPOS comprende le 17 categorie riportate nella Tabella 1.1 [3, 4]. Le caratteristiche morfologiche, come genere, numero e tempo verbale, sono rappresentate separatamente, quando applicabili, mediante gli attributi del campo `FEATS` [5].

| **Tag UPOS** | **Categoria grammaticale** | **Esempio contestuale** |
|---|---|---|
| `ADJ` | Aggettivo | Il gatto è **nero**. |
| `ADP` | Adposizione | Vado **a** casa. |
| `ADV` | Avverbio | Il gatto dorme **tranquillamente**. |
| `AUX` | Verbo ausiliare | Maria **ha** mangiato. |
| `CCONJ` | Congiunzione coordinante | Leggo **e** scrivo. |
| `DET` | Determinante | **Il** gatto dorme. |
| `INTJ` | Interiezione | **Oh**, sei arrivato! |
| `NOUN` | Nome comune | Il **gatto** dorme. |
| `NUM` | Numerale | Ho **tre** libri. |
| `PART` | Particella | John**'s** book. |
| `PRON` | Pronome | **Lei** legge un libro. |
| `PROPN` | Nome proprio | **Maria** legge un libro. |
| `PUNCT` | Punteggiatura | Sei a casa**?** |
| `SCONJ` | Congiunzione subordinante | Resto a casa **perché** piove. |
| `SYM` | Simbolo | Il risultato è 2 + 3. |
| `VERB` | Verbo | Il gatto **dorme**. |
| `X` | Altro | Ha pronunciato **xfgh**. |

Tabella 1.1: Categorie UPOS ed esempi contestuali. Per `PART` è illustrato il marcatore possessivo inglese; per `X`, una sequenza non interpretabile.

Per esempio, la frase «*Il gatto dorme.*» può essere rappresentata attraverso la sequenza `DET NOUN VERB PUNCT`, dove a ciascun token può essere associato un campo `FEATS` che ne specifica le caratteristiche morfologiche, e.g. per «*gatto*», etichettato come `NOUN`, contiene `Gender=Masc|Number=Sing` [5].

## 1.2 Le principali sfide del PoS tagging

Una delle principali difficoltà del PoS tagging è rappresentata dall'ambiguità grammaticale: una stessa parola può appartenere a più categorie, e la scelta del tag corretto dipende dal contesto [1]. Ad esempio, la parola *porta* può essere un nome (e.g. «*La porta è chiusa.*») o un verbo (e.g. «*Lui porta il libro.*»).

Un altro problema risiede nello sbilanciamento della distribuzione dei tag: alcune categorie, come nomi comuni (`NOUN`), determinanti (`DET`) e adposizioni (`ADP`), sono molto frequenti, mentre altre, come simboli (`SYM`) ed elementi non classificabili nelle altre categorie (`X`), compaiono raramente [6]. La presenza di classi con pochi esempi può rendere più difficile l'apprendimento di modelli robusti e generalizzabili [7].

A ciò si aggiungono la presenza di parole rare o mai osservate durante l'addestramento, come neologismi, termini specialistici e nomi propri, che il modello può quindi faticare a classificare [8]; le differenze tra il dominio lessicale dei testi di addestramento e quello dei testi da analizzare, che possono ridurre la capacità di generalizzazione del modello [1]; e gli eventuali errori o incoerenze nelle annotazioni, che possono influenzare sia l'apprendimento sia la valutazione [1].

Queste problematiche evidenziano l'importanza di adottare modelli capaci di costruire rappresentazioni linguistiche sensibili al contesto e di sfruttare conoscenze apprese da grandi quantità di dati testuali [9].

## 1.3 Modelli neurali per l'elaborazione del linguaggio

Naturalmente, il PoS tagging richiede modelli capaci di identificare le categorie grammaticali delle parole tenendo conto del contesto frasale. L'evoluzione delle tecniche di NLP ha progressivamente migliorato questa capacità, passando da rappresentazioni lessicali statiche ad architetture in grado di integrare informazioni contestuali sempre più complesse [2].

Tra gli approcci neurali più influenti, Word2Vec [10] ha permesso di apprendere rappresentazioni vettoriali dense delle parole, dette *word embeddings*, nelle quali termini utilizzati in contesti simili tendono ad assumere rappresentazioni vicine. Tali embedding sono tuttavia statici, i.e. associano a ciascuna parola un unico vettore indipendentemente dal contesto, rappresentando quindi un limite per il PoS tagging.

Le reti neurali ricorrenti (RNN), tra cui le Long Short-Term Memory (LSTM) [11], consentono invece di produrre rappresentazioni dipendenti dal contesto attraverso l'elaborazione sequenziale dei token. Le architetture bidirezionali, come le BiLSTM, consentono inoltre di integrare informazioni provenienti da entrambe le direzioni della sequenza [12, 8]. La natura ricorrente di questi modelli limita tuttavia la parallelizzazione e complica la gestione delle dipendenze a lungo raggio [13].

L'architettura Transformer [13] supera alcuni di questi limiti sostituendo la ricorrenza con meccanismi di attenzione. In particolare, la *Self-Attention* permette di modellare direttamente le relazioni tra i token, anche distanti tra loro, favorendo un'elaborazione parallela della sequenza e la produzione di rappresentazioni contestuali.

## 1.4 Architettura BERT

Tra i modelli basati sull'architettura Transformer, Bidirectional Encoder Representations from Transformers (BERT) [9] sfrutta un Transformer Encoder bidirezionale per produrre rappresentazioni contestuali dei token, risultando particolarmente adatto a compiti come il PoS tagging, nei quali la classificazione grammaticale dipende dal contesto frasale.

### 1.4.1 Tokenizzazione

L'input dell'encoder di BERT è costituito da token prodotti da WordPiece, un metodo di tokenizzazione che mantiene le parole intere oppure le suddivide in frammenti più piccoli, detti *subword*, selezionati da un vocabolario predefinito. Ad esempio, il tokenizer associato al modello `dbmdz/bert-base-italian-cased` rappresenta *bicicletta* con un singolo token, mentre suddivide *casetta* in `case` e `##tta`; il prefisso `##` indica la continuazione della parola. Questo approccio permette di rappresentare parole rare combinando frammenti già disponibili [9].

### 1.4.2 Struttura dell'encoder

Il modello `dbmdz/bert-base-italian-cased` adotta la configurazione BERT-Base, composta da 12 blocchi Transformer Encoder, 12 teste di attenzione per blocco e rappresentazioni di dimensione 768 [14]. Prima dell'elaborazione, gli embedding dei token vengono sommati a quelli posizionali e di segmento, che identificano rispettivamente la posizione nella sequenza e l'appartenenza ai segmenti di testo. Le rappresentazioni risultanti attraversano i 12 blocchi in successione, mantenendo invariata la loro dimensione [9].

Ogni blocco comprende una Multi-Head Self-Attention e una rete feed-forward. La prima esegue più operazioni di attenzione in parallelo, utilizzando proiezioni query, key e value apprese separatamente per ciascuna testa. Le uscite vengono concatenate e proiettate nuovamente nello spazio di dimensione 768 [13]. Una delle

particolarità dell'architettura è l'attenzione bidirezionale: ciascun token può utilizzare sia il contesto precedente sia quello successivo, senza vincoli causali [9].

Per ciascuna testa, l'attenzione è calcolata come:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}}\right)V, \tag{1.1}$$

dove $Q$, $K$ e $V$ sono proiezioni lineari apprese delle rappresentazioni in ingresso:

- **Query ($Q$):** rappresentano le informazioni ricercate da ciascun token nel contesto;
- **Key ($K$):** rappresentano le caratteristiche dei token utilizzate per calcolare la compatibilità con le query;
- **Value ($V$):** rappresentano le informazioni da aggregare mediante i pesi di attenzione.

Il termine $d_k$ indica la dimensionalità dei vettori key, pari a $768/12 = 64$ per ciascuna testa.

La rete feed-forward applica indipendentemente a ciascun token due trasformazioni lineari, da 768 a 3 072 componenti e viceversa, separate da un'attivazione GELU. Mentre l'attenzione integra informazioni tra posizioni, questa rete elabora le rappresentazioni individualmente. Entrambi i sottolivelli impiegano dropout per ridurre l'overfitting, oltre a connessioni residue e Layer Normalization per favorire la stabilità dell'addestramento [9, 13].

Al termine dei 12 blocchi, l'encoder restituisce un vettore contestuale di 768 componenti per ciascun token, utilizzabile dalle teste di classificazione dei compiti supervisionati.

### 1.4.3 Pre-addestramento

Nel BERT originale il preaddestramento combina il *Masked Language Modeling* (MLM) e la *Next Sentence Prediction* (NSP). Nel MLM viene selezionato casualmente il 15% dei token della sequenza: di questi, l'80% viene sostituito con il token speciale `[MASK]`, il 10% con un token casuale e il restante 10% rimane invariato. Il modello deve predire i token originali nelle posizioni selezionate, sfruttando il contesto bidirezionale. La NSP richiede invece di determinare se due segmenti di testo siano consecutivi nel corpus, mediante una classificazione binaria [9].

Attraverso il preaddestramento, BERT apprende rappresentazioni linguistiche contestuali da grandi quantità di testo, che possono successivamente essere adattate a diversi compiti supervisionati mediante fine-tuning.

Il modello `dbmdz/bert-base-italian-cased`, basato sull'architettura BERT-Base descritta in precedenza, è preaddestrato su corpora italiani e distingue tra lettere maiuscole e minuscole [14].

## 1.5 Obiettivi del progetto

Il presente lavoro studia l'adattamento di BERT al PoS tagging della lingua italiana, estendendo l'analisi anche alla predizione delle caratteristiche morfologiche `FEATS`. Si confrontano tre configurazioni: BERT congelato con teste di classificazione addestrabili, BERT sottoposto a fine-tuning completo e BERT fine-tuned con un Conditional Random Field (CRF) sulla sola sequenza UPOS. L'obiettivo principale è valutare il contributo del fine-tuning dell'encoder e della decodifica strutturata alla predizione dei tag UPOS. La predizione delle caratteristiche `FEATS` costituisce un obiettivo aggiuntivo.

Gli esperimenti utilizzano il treebank UD Italian-ISDT 2.15 e assumono parole già segmentate. UDPipe 2 costituisce il riferimento esterno per il confronto UPOS [6, 15]. Il protocollo sperimentale e le metriche di valutazione sono descritti nel capitolo dedicato alla metodologia (Capitolo 2).

# Capitolo 2

# Metodologia

Il protocollo sperimentale valuta l'adattamento di BERT alla predizione congiunta delle categorie UPOS e delle caratteristiche morfologiche FEATS. Le configurazioni condividono dati, tokenizzazione, inventari delle etichette e procedura di valutazione. Le differenze riguardano i parametri aggiornati durante l'addestramento e la decodifica della sequenza UPOS.

## 2.1 Disegno sperimentale

Si considerano tre configurazioni: un encoder congelato con teste lineari addestrabili, un encoder sottoposto a fine-tuning con le stesse teste e una variante fine-tuned con un CRF applicato alla sola uscita UPOS. Tutte utilizzano una testa UPOS e una testa per ciascun attributo FEATS. Il confronto tra le prime due configurazioni esamina l'adattamento dell'encoder con le rispettive impostazioni di ottimizzazione; il confronto tra le due varianti fine-tuned valuta l'introduzione delle transizioni UPOS. La condivisione dell'encoder consente a entrambi i compiti di contribuire all'apprendimento delle rappresentazioni [16].

| Configurazione | Encoder | UPOS | FEATS | Learning rate |
|---|---|---|---|---|
| BERT congelato | Pesi fissi | Lineare | Lineari | $10^{-3}$ |
| BERT fine-tuned | Addestrabile | Lineare | Lineari | $2 \cdot 10^{-5}$ |
| BERT + CRF | Addestrabile | Lineare e CRF | Lineari | $2 \cdot 10^{-5}$ |

Tabella 2.1: Configurazioni e learning rate. Le teste e l'eventuale CRF sono addestrabili in tutte le prove.

Ogni configurazione viene eseguita con i seed 42, 123 e 456, per un totale di nove prove. Ogni prova parte dal checkpoint preaddestrato originale e inizializza le proprie teste di classificazione. La ripetizione sugli stessi seed permette di osservare la variabilità dei risultati e di confrontare le configurazioni per coppie di esecuzioni [17].

## 2.2 Dati e annotazioni

### 2.2.1 Corpus e suddivisione

Il corpus impiegato è UD Italian-ISDT, release `r2.15`, con gli split ufficiali di training, sviluppo e test [6]. La Tabella 2.2 ne riporta le dimensioni. Il training viene utilizzato per aggiornare i parametri e costruire gli inventari delle classi; il development set guida la selezione del checkpoint; il test fornisce la valutazione finale.

| Split | Frasi | Parole | Tag UPOS presenti |
|---|---|---|---|
| Train | 13 121 | 276 014 | 17 |
| Dev | 564 | 11 907 | 17 |
| Test | 482 | 10 417 | 15 |

Tabella 2.2: Dimensioni degli split di UD Italian-ISDT 2.15. Le parole corrispondono alle righe CoNLL-U con identificatore intero.

La lettura dei file CoNLL-U conserva le parole sintattiche identificate da un ID intero. Le righe che descrivono token multiparola e nodi vuoti vengono escluse dalle sequenze etichettate [5]. Gli input linguistici sono le forme delle parole e i confini di frase del corpus. Il protocollo assume pertanto segmentazione gold e misura l'assegnazione delle etichette sulle medesime parole per tutte le configurazioni.

Gli split ufficiali sono mantenuti senza redistribuire le frasi. Un controllo delle sequenze di forme ha rilevato una frase identica tra training e sviluppo e nessuna sequenza identica tra training e test. Nel test, `INTJ` e `PART` hanno supporto zero, mentre `SYM` e `X` comprendono rispettivamente 5 e 13 parole. Queste caratteristiche motivano l'analisi delle prestazioni per categoria.

### 2.2.2 Rappresentazione di UPOS e FEATS

L'inventario UPOS comprende i 17 tag osservati nel training. Ogni parola riceve inoltre un vettore di etichette morfologiche, con una componente per ciascuno dei 14 attributi di training: `Clitic`, `Definite`, `Degree`, `Foreign`, `Gender`, `Mood`, `NumType`, `Number`, `Person`, `Polarity`, `Poss`, `PronType`, `Tense` e `VerbForm`. Gli inventari dei valori sono derivati esclusivamente dal training e comprendono complessivamente 40 coppie attributo-valore.

Ciascuna testa morfologica distingue i valori del proprio attributo e una classe tecnica `<NONE>`, che rappresenta l'assenza dell'attributo sulla parola. Per esempio, una parola con `Gender=Masc|Number=Sing` riceve i valori `Masc` e `Sing` nelle rispettive componenti e `<NONE>` per gli attributi assenti. Gli eventuali valori multipli dello stesso attributo sono ordinati e conservati come una singola etichetta composta, in accordo con la rappresentazione CoNLL-U [5].

Le annotazioni originali vengono conservate anche quando contengono attributi o valori estranei agli inventari appresi. Nel test compare, per esempio, `Typo=Yes`, assente dal training. Questo caso contribuisce alle metriche morfologiche e viene

conteggiato come errore quando non è predetto. Nei tensori ausiliari destinati alla loss, i valori sconosciuti delle teste esistenti sono ricondotti a `<NONE>`; il calcolo delle metriche utilizza le annotazioni originali.

## 2.3 Preprocessing e allineamento

Si utilizza il tokenizer fast associato a `dbmdz/bert-base-italian-cased`, applicato alle parole presegmentate mediante `is_split_into_words=True`. La funzione `word_ids()` collega ciascuna subword alla parola di origine [18]. Il vettore delle etichette UPOS e FEATS viene assegnato soltanto alla prima subword di ogni parola. Token speciali e subword successive ricevono il valore tecnico $-100$, utilizzato per escluderli dalla loss di classificazione e dalle metriche.

Il padding è dinamico: ciascun batch viene esteso alla lunghezza della frase tokenizzata più lunga che contiene. Anche le posizioni di padding ricevono $-100$. Una maschera individua le prime subword; nella variante CRF gli score in queste posizioni vengono raccolti in sequenze continue di parole. La maschera identifica le posizioni valide, mentre i valori delle etichette vengono utilizzati come supervisione della funzione obiettivo.

La lunghezza massima è fissata a 512 token, inclusi quelli speciali. Il preprocessing verifica che ogni parola abbia una sola posizione supervisionata e interrompe l'esecuzione quando una frase supera il limite. Le lunghezze massime osservate sono 332 token nel training, 110 nello sviluppo e 133 nel test. Tutte le frasi vengono quindi elaborate interamente.

## 2.4 Modelli e funzione obiettivo

L'encoder condiviso produce le rappresentazioni utilizzate dalle 15 teste: una per UPOS e 14 per FEATS. Ogni testa applica una trasformazione lineare alla rappresentazione della prima subword dopo un dropout con probabilità 0.1. Le classi vengono predette tramite lo score massimo; la corrispondente loss è la cross-entropy mediata sulle parole valide [18].

Nella configurazione con CRF, gli score della testa UPOS costituiscono le emissioni di una catena lineare. Il training minimizza la log-verosimiglianza negativa della sequenza corretta, normalizzata sul numero di parole del batch mediante la riduzione `token_mean`. La decodifica Viterbi seleziona la sequenza UPOS con score massimo [19, 20]. Le teste FEATS mantengono loss di classificazione e decodifica per parola.

Indicando con $\mathcal{A}$ l'insieme dei 14 attributi morfologici, la loss complessiva è

$$\mathcal{L} = \mathcal{L}_{\text{UPOS}} + \lambda \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \mathcal{L}_a, \qquad \lambda = 1. \tag{2.1}$$

La media delle loss FEATS mantiene il peso complessivo del compito morfologico indipendente dal numero delle teste. Il coefficiente $\lambda$ è fissato a 1 per tutte le configurazioni. Anche la classe `<NONE>` partecipa all'apprendimento, consentendo di predire quali attributi siano presenti su ciascuna parola.

## 2.5 Procedura di addestramento

L'ottimizzazione utilizza AdamW [21], attraverso il `Trainer` di Transformers [22]. Le impostazioni comuni sono riportate nella Tabella 2.3; i learning rate delle singole configurazioni sono indicati nella Tabella 2.1. Il valore più elevato della variante congelata riguarda soltanto le teste, mentre nelle due varianti fine-tuned viene aggiornato anche l'encoder. I learning rate e il peso dei compiti rimangono fissi per tutti i seed.

| Parametro | Valore |
|---|---|
| Epoche | 5 |
| Batch di training | 8 frasi |
| Batch di valutazione | 16 frasi |
| Weight decay | 0.01 |
| Warmup | 10% dei passi |
| Clipping della norma dei gradienti | 1.0 |
| $\beta_1$, $\beta_2$ di AdamW | 0.9, 0.999 |
| $\epsilon$ di AdamW | $10^{-8}$ |

Tabella 2.3: Impostazioni di ottimizzazione condivise. Il learning rate cresce durante il warmup e decresce linearmente nei passi successivi.

Il seed viene impostato prima della creazione di ogni modello e controlla l'inizializzazione delle teste, l'ordine dei batch e le operazioni stocastiche. Nella variante congelata i parametri dell'encoder restano fissi. Nelle altre configurazioni vengono aggiornati congiuntamente encoder, teste e, quando presente, CRF.

Le teste lineari sono inizializzate secondo la procedura BERT. Le transizioni iniziali, finali e interne del CRF vengono inizializzate uniformemente nell'intervallo $[-0.1, 0.1]$, come previsto da `pytorch-crf` [20]. L'inizializzazione del modulo CRF è esplicitamente integrata nel caricamento del modello preaddestrato e verificata prima dell'addestramento. Un controllo durante l'esecuzione interrompe la prova in presenza di loss non finite.

Quando CUDA è disponibile, viene utilizzata la precisione mista FP16. L'ambiente GPU del progetto impiega una NVIDIA GeForce RTX 4070 SUPER con 12 GB di memoria. La gestione della memoria libera i modelli al termine di ogni prova, consentendo l'esecuzione sequenziale delle configurazioni.

## 2.6 Selezione del checkpoint

Al termine di ogni epoca il modello viene valutato sul development set e viene salvato un checkpoint. La selezione massimizza il punteggio

$$S_{\text{dev}} = \frac{1}{2}\left(F_{1,\text{UPOS}}^{\text{macro,tot}} + F_{1,\text{FEATS}}^{\text{macro,tot}}\right), \tag{2.2}$$

dove entrambe le macro-F1 sono calcolate sugli inventari totali definiti nella sezione 2.7. Questa scelta assegna lo stesso peso ai due compiti e rende la selezione sensibile anche alle classi meno frequenti.

Il checkpoint con il valore massimo di $S_{\text{dev}}$ viene ricaricato e salvato come modello della prova. La valutazione sul test viene eseguita dopo il completamento delle nove prove e delle rispettive selezioni sul development set. Ogni seed contribuisce alle statistiche finali della propria configurazione.

## 2.7 Valutazione

Le predizioni vengono ricondotte alle parole originali attraverso l'allineamento delle prime subword. L'ordine delle parole e le etichette UPOS di riferimento sono verificati rispetto alle annotazioni dello split valutato. Tutte le configurazioni vengono confrontate sulle stesse unità linguistiche.

### 2.7.1 Metriche UPOS

L'accuracy UPOS è la frazione di parole con categoria grammaticale corretta. Per ciascun tag $k$ si calcolano veri positivi, falsi positivi e falsi negativi, dai quali si ricavano precision, recall e F1. La macro-F1 è la media non pesata degli F1 delle categorie incluse:

$$F_{1,k} = \frac{2TP_k}{2TP_k + FP_k + FN_k}, \qquad F_1^{\text{macro}} = \frac{1}{|C|} \sum_{k \in C} F_{1,k}. \tag{2.3}$$

Le divisioni con denominatore nullo producono F1 pari a zero. La versione *totale* considera i 17 tag del training; quella sulle classi *presenti* considera i tag con supporto gold nello split valutato. Nel test questi ultimi sono 15, poiché `INTJ` e `PART` sono assenti. Le due convenzioni permettono di distinguere la copertura dell'inventario dalle prestazioni sulle categorie effettivamente osservate.

### 2.7.2 Metriche FEATS

L'accuracy FEATS principale richiede che l'intero insieme di coppie attributo-valore di una parola coincida con quello gold. Indicando con $G_i$ e $P_i$ gli insiemi corretto e predetto per la parola $i$, si calcola

$$\text{Acc}_{\text{FEATS}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[G_i = P_i]. \tag{2.4}$$

Anche gli insiemi vuoti partecipano alla valutazione. Una misura aggiuntiva, l'accuracy per attributo, conta le decisioni corrette sulle coppie parola-attributo e include l'assenza rappresentata da `<NONE>`. L'insieme degli attributi valutati comprende quelli del training e quelli eventualmente presenti nelle annotazioni gold o nelle predizioni dello split. Questa misura viene interpretata insieme all'accuracy dell'insieme completo, poiché molte decisioni riguardano attributi assenti.

La macro-F1 FEATS tratta ciascuna coppia attributo-valore, per esempio `Gender=Fem`, come una classe. Veri positivi, falsi positivi e falsi negativi sono conteggiati sulla presenza della coppia negli insiemi delle singole parole. La classe tecnica `<NONE>` è esclusa da questa media, così che l'abbondanza di attributi assenti non domini la misura.

La macro-F1 *totale* comprende le 40 coppie del training e le eventuali ulteriori coppie osservate nelle annotazioni gold o nelle predizioni dello split valutato. La macro-F1 sulle classi *presenti* comprende soltanto le coppie con supporto gold positivo. Le coppie sconosciute al modello rimangono quindi nel conteggio degli errori. Le metriche utilizzano le annotazioni morfologiche originali; la loss sul test non viene impiegata come indicatore di prestazione, poiché i tensori delle etichette rappresentano soltanto gli inventari delle teste.

Si riporta inoltre l'accuracy congiunta, definita come la frazione di parole per le quali sono corretti sia UPOS sia l'intero insieme FEATS. I file per attributo permettono di esaminare separatamente accuracy e macro-F1 dei singoli attributi morfologici, includendo la rispettiva classe di assenza.

### 2.7.3 Aggregazione e analisi degli errori

Per ogni metrica vengono calcolate media e deviazione standard campionaria sui tre seed [17]. Le misure delle singole prove vengono conservate per rendere verificabili gli scarti tra configurazioni eseguite con lo stesso seed. I risultati numerici sono salvati come frazioni e visualizzati nelle tabelle in percentuale.

L'analisi degli errori comprende le principali confusioni UPOS, sommate sui tre seed, e le coppie FEATS con il maggior numero complessivo di falsi positivi e falsi negativi. Per ogni prova sono disponibili anche precision, recall, F1 e supporto delle singole classi, insieme alle predizioni e agli errori a livello di parola.

## 2.8 Confronto con UDPipe 2

Il riferimento esterno UPOS è il modello ufficiale `italian-isdt-ud-2.15-241121`, disponibile tramite il servizio UDPipe 2 [15]. Le predizioni di riferimento sono state ottenute fornendo le forme delle parole e i confini gold del test, con le annotazioni grammaticali rimosse dall'input. Il confronto verifica identità del modello, hash del test e corrispondenza di indice di frase, indice di parola, forma ed etichetta gold.

Sulle predizioni salvate vengono ricalcolate accuracy e macro-F1 UPOS con le stesse convenzioni adottate per BERT. Il punteggio UPOS F1 pubblicato per la tokenizzazione gold è 98,75% [15]. In presenza di un allineamento completo delle parole, tale F1 coincide numericamente con l'accuracy. La macro-F1 per categoria viene invece ricavata dalle predizioni individuali.

Il confronto esterno riguarda UPOS, per il quale sono disponibili le predizioni di riferimento nel notebook. Le misure FEATS descrivono il confronto tra le tre configurazioni BERT. UDPipe 2 utilizza una propria architettura e rappresentazioni

contestuali da BERT multilingue; i risultati confrontano pertanto sistemi completi nelle medesime condizioni di input [15].

## 2.9 Riproducibilità e integrità delle esecuzioni

Modello e tokenizer utilizzano la stessa revisione del checkpoint, identificata dal commit

`843e404d71f4c63972443e3463456f59024db278`.

Gli hash SHA-256 dei file CoNLL-U identificano i dati effettivamente impiegati. I metadati registrano inventari, configurazioni, seed, iperparametri e versioni dei pacchetti. L'ambiente di esecuzione utilizza Python 3.13.3, PyTorch 2.6.0 con CUDA 12.4 e Transformers 4.46.3; le versioni effettive sono riportate nei file dell'esperimento.

Gli artefatti vengono raccolti in `outputs/bert_upos_feats_comparison_v1`, con una cartella per ogni configurazione e seed. Ogni prova salva il modello selezionato, la cronologia di training, i metadati, le predizioni per parola, le metriche complessive, le misure per tag e attributo e i casi errati. I riepiloghi aggregati vengono ricostruiti dalle prove della campagna corrente.

Il riuso di una prova completata richiede la corrispondenza della configurazione salvata. Per i modelli CRF viene verificata anche la versione della procedura di inizializzazione; i checkpoint precedenti vengono archiviati prima di ripetere la prova con l'inizializzazione corretta. Questo controllo mantiene distinti i risultati delle esecuzioni valide dagli artefatti prodotti con parametri iniziali incompatibili. Hardware, precisione numerica e versioni delle librerie costituiscono ulteriori condizioni per riprodurre l'esperimento.

# Bibliografia


[1] Christopher D. Manning. Part-of-speech tagging from 97% to 100%: Is it time for some linguistics? In *Computational Linguistics and Intelligent Text Processing*, volume 6608 of *Lecture Notes in Computer Science*, pages 171–189. Springer, 2011. URL: https://nlp.stanford.edu/~manning/papers/CICLing2011-manning-tagging.pdf, doi:10.1007/978-3-642-19400-9_14.

[2] Yoav Goldberg. A primer on neural network models for natural language processing. *Journal of Artificial Intelligence Research*, 57:345–420, 2016. doi:10.1613/jair.4992.

[3] Joakim Nivre, Marie-Catherine de Marneffe, Filip Ginter, Jan Hajič, Christopher D. Manning, Sampo Pyysalo, Sebastian Schuster, Francis Tyers, and Daniel Zeman. Universal Dependencies v2: An evergrowing multilingual treebank collection. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 4034–4043. European Language Resources Association, 2020. URL: https://aclanthology.org/2020.lrec-1.497/.

[4] Universal Dependencies contributors. Universal POS tags. Documentazione di Universal Dependencies v2. Consultato il 7 novembre 2026. URL: https://universaldependencies.org/u/pos/.

[5] Universal Dependencies contributors. CoNLL-U format. Specifiche di Universal Dependencies v2. Consultato il 7 novembre 2026. URL: https://universaldependencies.org/format.html.

[6] Universal Dependencies contributors. UD Italian-ISDT. Treebank, release r2.15, 2024. Consultato il 7 novembre 2026. URL: https://github.com/UniversalDependencies/UD_Italian-ISDT/tree/r2.15.

[7] Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. doi:10.1109/TKDE.2008.239.

[8] Barbara Plank, Anders Søgaard, and Yoav Goldberg. Multilingual part-of-speech tagging with bidirectional long short-term memory models and auxiliary loss. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 412–418. Association for Computational Linguistics, 2016. URL: https://aclanthology.org/P16-2067/, doi:10.18653/v1/P16-2067.

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186. Association for Computational Linguistics, 2019. URL: https://aclanthology.org/N19-1423/, doi:10.18653/v1/N19-1423.

[10] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013. URL: https://papers.nips.cc/paper_files/paper/2013/hash/9aa42b31882ec039965f3c4923ce901b-Abstract.html.

[11] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi:10.1162/neco.1997.9.8.1735.

[12] Mike Schuster and Kuldip K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681, 1997. doi:10.1109/78.650093.

[13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL: https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[14] DBMDZ. BERT base italian cased. Model card, Hugging Face. Revisione `843e404d71f4c63972443e3463456f59024db278`. Consultato il 7 novembre 2026. URL: https://huggingface.co/dbmdz/bert-base-italian-cased.

[15] ÚFAL, Charles University. UDPipe 2 models: Universal Dependencies 2.15. Documentazione dei modelli. Modello `italian-isdt-ud-2.15-241121`. Consultato il 7 novembre 2026. URL: https://ufal.mff.cuni.cz/udpipe/2/models.

[16] Dan Kondratyuk and Milan Straka. 75 languages, 1 model: Parsing Universal Dependencies universally. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2779–2795. Association for Computational Linguistics, 2019. URL: https://aclanthology.org/D19-1279/, doi:10.18653/v1/D19-1279.

[17] Nils Reimers and Iryna Gurevych. Reporting score distributions makes a difference: Performance study of LSTM-networks for sequence tagging. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 338–348. Association for Computational Linguistics, 2017. URL: https://aclanthology.org/D17-1035/, doi:10.18653/v1/D17-1035.

[18] Hugging Face. Token classification. Documentazione di Transformers, versione 4.46.3. Consultato il 7 novembre 2026. URL: https://huggingface.co/docs/transformers/v4.46.3/tasks/token_classification.

[19] John Lafferty, Andrew McCallum, and Fernando C. N. Pereira. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of the Eighteenth International Conference on Machine Learning*, pages 282–289. Morgan Kaufmann, 2001. URL: https://repository.upenn.edu/cis_papers/159.

[20] pytorch-crf contributors. pytorch-crf: Conditional random fields in PyTorch. Documentazione del software, versione 0.7.2. Consultato il 7 novembre 2026. URL: https://pytorch-crf.readthedocs.io/en/stable/.

[21] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL: https://arxiv.org/abs/1711.05101.

[22] Hugging Face. Trainer. Documentazione di Transformers, versione 4.46.3. Consultato il 7 novembre 2026. URL: https://huggingface.co/docs/transformers/v4.46.3/en/main_classes/trainer.